Machine Learning Project Report

UCI Heart Disease (Classification) and Online News Popularity (Regression)

Student List:

Full name	Group	Section
ABDELI Zohra	G1	S1
HADJ MESSAOUD Maria	G7	S2
BENBOUABDELLAH Nadia	G6	S2
KHALDI Aya	G6	S2

May 17, 2025

Note: The length of this report exceeded 15 pages primarily due to the inclusion of numerous figures and visualizations.

Contents

1 UCI Heart Disease (Classification) 3

1.1 Abstract . . . 3
1.2 Introduction . . . 3
1.3 Dataset Description . . . 3
1.4 Methodology . . . 4
1.5 Results and Analysis . . . 5
1.6 Discussion . . . 8
1.7 Conclusion . . . 8

2 Online News Popularity (Regression) 9

2.1 Abstract . . . 9
2.2 Introduction . . . 9
2.3 Data Description . . . 9
2.4 Exploratory Data Analysis . . . 9
2.5 Methodology . . . 10
2.5.1 Data Preprocessing and Feature Engineering . . . 10
2.5.2 Data Splitting . . . 12
2.5.3 Model Selection and Training . . . 12
2.5.4 Model Evaluation . . . 12
2.6 Results . . . 14
2.6.1 Performance on Transformed Scales . . . 15
2.6.2 Performance on Original Scale (after Transformation) . . . 15
2.6.3 Hyperparameter Tuning for SVR . . . 18
2.7 Conclusion . . . 19

References 20

Who did what in the project? 20

1 UCI Heart Disease (Classification)

1.1 Abstract

Heart disease remains one of the leading causes of mortality worldwide, highlighting the urgent need for effective diagnostic tools. This project investigates the use of machine learning techniques to predict the presence of heart disease based on patient medical data from the UCI Heart Disease dataset. The dataset presents several challenges, including missing values and variability in data quality, which necessitate careful preprocessing and model selection. A range of classification algorithms were explored to understand their applicability to medical prediction tasks and to assess their ability to generalize across different data conditions. The study aims to contribute to the growing body of work on data-driven approaches in healthcare and to demonstrate how machine learning can aid in early detection and decision support.

1.2 Introduction

Heart disease is a term used to describe a range of conditions that affect the heart, including coronary artery disease (CAD). CAD is the most common, impairing blood flow to the heart and often remaining undiagnosed until severe symptoms such as chest pain, fatigue, or shortness of breath occur. Heart disease is frequently used interchangeably with cardiovascular disease, which refers to conditions involving narrowed or blocked blood vessels that can lead to heart attacks, or strokes. Globally, heart disease is a leading cause of morbidity and mortality, accounting for about 610,000 deaths annually in the United States alone—approximately one in every four deaths. The complexity of diagnosing heart disease arises from the interplay of multiple risk factors like diabetes, high blood pressure, high cholesterol, and abnormal pulse rate. In recent years, the healthcare industry has generated vast amounts of clinical data, making traditional diagnostic methods increasingly insufficient. This has led researchers to turn to machine learning techniques to uncover hidden patterns and make accurate predictions. ML has proven effective in analyzing large healthcare datasets, enabling timely and informed decision-making. In this project, we will apply and compare ML algorithms to classify whether an individual is suffering from heart disease, using the widely studied Heart Disease dataset from the UCI Machine Learning Repository.

1.3 Dataset Description

The UCI Heart Disease dataset contains information on patient attributes and a target variable indicating the presence and severity of heart disease. The key features are described below:

Demographic & Baseline Health

- **Age**: Age in years. Risk increases with age.
- **Sex**: 1 = Male, 0 = Female.

Symptoms & Clinical Presentation

- **Chest Pain Type (cp)**:
  1. Typical angina (heart-related)
  2. Atypical angina
  3. Non-anginal pain
  4. Asymptomatic (silent heart issues)

Clinical Measurements

- **Resting Blood Pressure (trestbps)**: High values may indicate hypertension.
- **Cholesterol (chol)**: High cholesterol is linked to artery blockage.
- **Fasting Blood Sugar (fbs)**: > 120 mg/dL (1 = true) suggests diabetes risk.
- **Resting ECG (restecg)**:
  1. Normal
  2. ST-T abnormality
  3. Left ventricular hypertrophy
- **Max Heart Rate (thalach)**: Lower values may show poor heart fitness.
- **Exercise-Induced Angina (exang)**: 1 = Yes, indicates pain from exertion.
- **Oldpeak**: ST depression after exercise; higher = more stress on heart.
- **Slope**:
  1. Upsloping (better)
  2. Flat
  3. Downsloping (riskier)

Advanced Diagnostic Results

- **ca**: Number of major blocked vessels (0–3). More blockages = more risk.
- **thal**:
  1. Normal
  2. Fixed defect (scarring)
  3. Reversible defect (poor blood flow)

Note: The values for 'thal' are 3, 6, 7 in the text, which correspond to the descriptions above (3: Normal, 6: Fixed defect, 7: Reversible defect). The original dataset might have used different numeric encodings.

Target Variable

- **num**: Heart disease severity
  - 0: No disease
  - 1–3: Increasing levels of severity (mild to severe)
  - 4: Severe disease

Note: The target was simplified to binary (0: No, 1: Yes) in the methodology.

1.4 Methodology

The methodology for the UCI Heart Disease classification project involved careful data handling and model evaluation.

Data Source and Selection

After analyzing datasets from sources like Switzerland and Cleveland, it was found that most files contained significant missing values. Only the Cleveland dataset showed reasonable consistency. Two main approaches were therefore adopted:

1. Use only the Cleveland dataset, which has minimal missing data, to train models including KNN, Naive Bayes, Logistic Regression, ANN, SVM, Decision Tree, and Random Forest.
2. Apply a custom K-Nearest Neighbors (KNN) imputation to fill in missing values across the full dataset (combining data from multiple sources) and retain more samples for training. The imputation used 5 nearest neighbors, taking the average for continuous features (e.g., `trestbps`, `chol`, `thalach`, `oldpeak`) and majority vote for categorical ones (e.g., `sex`, `cp`, `slope`, `ca`, `thal`).

To assess the quality of the imputation, feature distributions were examined before and after the process. This allowed for a more complete and representative dataset for modeling.

Data Preprocessing

For the selected dataset (either Cleveland only or the imputed full dataset), the following preprocessing steps were applied:

- **Handling Missing Values**: In Approach 1, rows with missing values in the Cleveland dataset (? in `ca` and `thal`) were dropped. In Approach 2, imputation methods were used as described above.
- **Data Type Conversion**: Features were converted to appropriate data types (e.g., (`ca`, `thal`) were cast to integer types after cleaning).
- **Target Simplification**: The target variable was converted into a binary format: 0 for no disease, and 1 for any level of disease (1-4) to simplify the classification task.
- **Feature Scaling**: Numerical features were standardized using `StandardScaler` to improve model performance, especially for distance-based classifiers.
- **Encoding Categorical Variables**: Label encoding was applied to discrete variables when necessary.

Model Implementation and Evaluation

Several baseline and advanced machine learning models were evaluated:

- **Baseline Models**: Logistic Regression, Naive Bayes, SVM, Decision Tree, Random Forest, K-Nearest Neighbors (KNN), and MLP (ANN)
- **Custom Pipelines**: For KNN and others, pipelines were built to integrate scaling and model training
- **Hybrid Models**: A Naive Bayes hybrid combining `GaussianNB` (for continuous features) and `CategoricalNB` (for discrete features) was implemented to handle mixed-type data effectively

Models were evaluated using the following criteria:

- Accuracy on train and test sets
- Cross-validation scores (5-fold)
- Confusion matrices
- Classification reports (precision, recall, F1-score)

Hyperparameter Tuning

Grid search and randomized search were used for hyperparameter optimization:

- `GridSearchCV`: Applied to models like Decision Trees, KNN, and Logistic Regression
- `RandomizedSearchCV`: Used for SVM due to its larger hyperparameter space

Each model's optimal parameters were selected based on cross-validation accuracy, and the best model configurations were then used for final evaluation on the test set.

1.5 Results and Analysis

Analysis of KNN Imputation Impact on Dataset Features

Statistical Comparison: Before vs. After Imputation

Impact Assessment of KNN Imputation

- **Feature Type**: Categorical, Continuous
- **Features**: `sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, `thal` (Categorical); `age`, `trestbps`, `chol`, `thalach`, `oldpeak` (Continuous)
- **Imputation Method**: Majority vote (5-NN) for categorical, Mean of 5-NN for continuous
- **Impact on Distribution**:

Table 1: Statistical Comparison of Features Before and After KNN Imputation

Feature	Before Imputation		After Imputation		Change	
	Mean	Std Dev	Mean	Std Dev	Mean	Std Dev
`trestbps`	132.11	19.32	132.07	18.75	-0.04 ↓	-0.57 ↓
`chol`	192.42	114.85	194.01	113.09	+1.59 ↑	-1.76 ↓
`thalach`	135.39	25.90	136.02	25.33	+0.63 ↑	-0.57 ↓
`oldpeak`	0.86	1.09	0.88	1.06	+0.02 ↑	-0.03 ↓

  - Categorical: Not quantified in statistical terms in the provided text.
  - Continuous: Minimal impact on central tendencies and variance.

Key Observations

- **Preservation of Distributions**: All continuous features maintained their statistical properties with minimal changes. Standard deviations decreased slightly, suggesting a minor smoothing effect. Mean values remained stable with changes less than 1% for most features.
- **Imputation Quality Indicators**: The minimal changes to distributions suggest the KNN imputation preserved the underlying data structure. The consistent pattern of slightly decreased standard deviations indicates a minor regularization effect, which is expected and appropriate with KNN imputation.

Baseline Modeling Overview

For both approaches (Cleveland only and Imputed), a set of baseline classification models were initially evaluated using their default parameters without hyperparameter tuning, feature scaling, or advanced preprocessing. This baseline evaluation serves to:

- Provide a straightforward reference point for model performance.
- Identify any immediate strengths or weaknesses of each algorithm on the datasets.
- Detect signs of overfitting or underfitting early in the process.
- Guide future steps in model refinement and optimization.

Table 2: Baseline Model Performance Comparison

Model	Train Acc (Cleveland)	Test Acc (Cleveland)	Train Acc (Imputed)	Test Acc (Imputed)	Notes
Random Forest	100%	77%	100%	82%	Overfits in both; test performance improved with imputed data
Decision Tree	100%	73%	100%	74%	Overfits heavily; pruning or regularization needed
Naive Bayes	86%	85%	81.36%	86%	Consistent strong performer across both approaches
Logistic Regression	86%	82%	81.85%	85%	Slightly better generalization with more data after imputation
SVM (Linear)	86%	83%	81.23%	86%	Improved balanced accuracy with larger imputed dataset
KNN	76%	63%	79.26%	67%	Underperforms both times; scaling and tuning needed
ANN	83%	77%	82.72%	82%	Benefits from more data; recall for minority class improves

Observations from Baseline Models:

- The Cleveland dataset provides a clean baseline with minimal missing values, allowing models to perform well without complex preprocessing.
- The imputation approach using the other three datasets increased the amount of data available for training and testing, resulting in generally improved test performance, especially for SVM, Logistic Regression, and Random Forest.
- Naive Bayes remained the most stable and high-performing model in both cases, demonstrating strong generalization capability.
- Tree-based models (Decision Tree and Random Forest) exhibit consistent overfitting, highlighting the need for pruning or regularization regardless of dataset size or preprocessing.
- KNN struggles in both approaches, likely due to sensitivity to feature scales and hyperparameters, requiring further tuning.
- ANN showed improved performance with the larger imputed dataset, suggesting that it benefits from more training data despite its sensitivity to hyperparameters.

Comparative Study of Optimized Models on Clean and Imputed Datasets

After baseline evaluation, hyperparameter tuning and scaled data were used to optimize model performance.

1. **K-Nearest Neighbors (KNN)** Cross-Validation Accuracy dropped from 86.47% to 79.63%, suggesting better generalization prior to imputation. Test Accuracy slightly decreased (85.00% $\rightarrow$ 83.00%), likely due to shifts in feature distributions post-imputation. *Interpretation*: Imputation may have reduced beneficial sparsity or introduced noise, affecting KNN's local decision logic.
2. **Random Forest** Test Accuracy improved (83.33% $\rightarrow$ 85%), with a slight drop in training accuracy indicating reduced overfitting. Model Complexity increased (estimators: 70 $\rightarrow$ 510), reflecting the need for deeper pattern extraction. *Interpretation*: Imputation enhanced generalization by providing more consistent training data.
3. **Support Vector Machine (SVM)** Test Accuracy improved (83.33% $\rightarrow$ 87.00%), while training accuracy dropped slightly. Model shifted from a linear kernel to RBF, and regularization was relaxed (C increased). Class weight was adjusted to "balanced" post-imputation. *Interpretation*: Imputation introduced patterns requiring more flexible decision boundaries, improving generalization.
4. **Naive Bayes** Test Accuracy rose slightly (83.33% $\rightarrow$ 84.57%). Training Accuracy: GaussianNB improved; CategoricalNB declined. Class-wise behavior flipped: Recall improved for class 1 post-imputation. *Interpretation*: Imputation altered class distribution and feature behavior, benefiting GaussianNB more overall.
5. **Logistic Regression** Test Accuracy increased notably (83.33% $\rightarrow$ 88.00%), but AUC decreased slightly (0.93 $\rightarrow$ 0.9166). Training Accuracy and CV Score declined, suggesting more regularization (C: 0.1 $\rightarrow$ 0.01) was needed to mitigate new noise. *Interpretation*: Larger dataset post-imputation improved generalization despite slightly noisier training dynamics.
6. **Artificial Neural Network (ANN)** Test Accuracy slightly dropped (83.33% $\rightarrow$ 82.00%), though validation accuracy improved. Recall decreased for class 1, improved for class 0 — indicating a shift in sensitivity. AUC remained high (from 0.93 to 0.9166), suggesting minor degradation in overall discrimination. *Interpretation*: Imputation preserved most predictive power; shifts in recall highlight class-specific impacts.

Overall Insights from Optimized Models:

- **Imputation Benefits**: Larger, more complete datasets improved test performance for most models (especially SVM, RF, Logistic Regression).
- **Trade-offs**: Slight declines in training accuracy, AUC, or cross-validation scores hint at added complexity or noise. **Best Improvement**: Logistic Regression and SVM showed the clearest gains in generalization.

- **Most Affected**: KNN saw the greatest performance drop, likely due to altered feature neighborhoods.

1.6 Discussion

The comparative analysis of optimized models on clean (Cleveland dataset) vs imputed datasets reveals several important insights. Overall, imputation generally enhanced test performance and model generalization, particularly for models like Support Vector Machine (SVM), Random Forest, and Logistic Regression. These models benefited from the increased data completeness and consistency post-imputation, allowing them to capture more complex patterns and reduce overfitting.

However, this improvement came with some trade-offs. Training accuracy and cross-validation scores for most models slightly declined after imputation, suggesting the introduction of noise or increased data complexity. Logistic Regression, for instance, required stronger regularization to manage this added noise, balancing bias and variance more effectively.

K-Nearest Neighbors (KNN) was the most negatively affected model, experiencing notable drops in both cross-validation and test accuracy. This is likely due to imputation altering local feature distributions, which disrupted KNN's reliance on neighborhood proximity for classification.

Artificial Neural Networks (ANN) showed mixed results with a slight drop in test accuracy and a shift in recall scores between classes, indicating class-specific sensitivity changes after imputation. Naive Bayes models displayed subtle performance shifts, reflecting changes in feature and class distributions.

Compared to existing public implementations, such as the notebook by g-shreekant on GitHub, our optimized models achieved slightly better performance metrics. This reflects the value of systematic preprocessing, model tuning, and the inclusion of imputed data in improving predictive capability. Our work demonstrates the importance of refining baseline approaches to achieve stronger and more generalizable results.

Future work should explore alternative imputation strategies, such as multiple imputation or deep learning-based methods, to assess their impact on model robustness. Incorporating domain knowledge for feature engineering, and validating model performance on entirely separate datasets, would further strengthen the conclusions.

1.7 Conclusion

This project conducted a comprehensive analysis of heart disease prediction using machine learning models on both clean and imputed datasets. Initially, baseline models were evaluated on the Cleveland dataset to establish reference performance levels. Subsequently, advanced preprocessing steps—including imputation, scaling, and hyperparameter tuning—were applied across all four datasets to assess the impact on model performance.

Key findings highlight that models such as Support Vector Machine, Logistic Regression, and Random Forest benefited significantly from imputation and optimization, showing improved test accuracy and generalization. In contrast, K-Nearest Neighbors exhibited reduced performance post-imputation, underscoring the importance of dataset structure for distance-based methods.

The project demonstrates that data completeness and thoughtful preprocessing play a crucial role in the effectiveness of machine learning models in medical prediction tasks. By evaluating multiple algorithms under varying conditions, the study offers valuable insights for selecting and optimizing models for clinical decision support systems.

2 Online News Popularity (Regression)

2.1 Abstract

This report details a machine learning project focused on predicting the popularity of online news articles, as measured by the number of social network shares, the project explores various regression algorithms to model the complex relationship between article features and share counts. Key challenges identified include the highly skewed distribution of the features and the presence of outliers. Despite comprehensive preprocessing techniques and the application of multiple regression models, achieving accurate prediction of the exact number of shares proved challenging. The findings highlight the inherent difficulty of this regression task with the given data characteristics, suggesting potential limitations for precise popularity forecasting using standard regression approaches.

2.2 Introduction

In the digital age, understanding and predicting the popularity of online content is crucial for publishers, marketers, and content creators. The ability to forecast which articles are likely to gain traction can inform content strategy, distribution efforts, and resource allocation. This project investigates the application of machine learning techniques to predict the popularity of online news articles based on their features.

2.3 Data Description

This project focused on predicting the popularity of online news articles, measured by the number of social network shares (`shares`), using the Online News Popularity dataset. Sourced from Mashable articles published over two years, the dataset contains 39,797 instances and 58 predictive features. These features capture diverse aspects of each article, including content characteristics, metadata, multimedia elements, keyword performance, publication timing, latent topics derived from LDA, and sentiment/subjectivity scores. The `url` and `timedelta` features were identified as non-predictive and excluded from the analysis. Features are a mix of integer and real data types.

2.4 Exploratory Data Analysis

Exploratory Data Analysis (EDA) was conducted to understand the dataset's structure and key characteristics. Features were conceptually grouped into categories such as textual, multimedia, keyword, content type, self-reference, weekday, LDA topics, and subjectivity/polarity features to facilitate analysis.

Beyond the statistical analysis, initial exploration yielded several qualitative insights into factors potentially influencing popularity. It was observed that both the number and quality of keywords appear relevant, with more and better keywords correlating with increased shares. Titles of a moderate word count seemed more likely to be shared than very short ones. Visual content in the form of images appeared more associated with shares than videos, possibly reflecting user preference. Observations on reader behavior suggested Tuesday and Wednesday as popular days for consuming articles, potentially linked to weekly routines.

Analysis of the target variable, `shares`, revealed a highly skewed distribution. A large majority of articles have a low number of shares, while a small subset receives significantly higher share counts, forming a long tail. This severe positive skewness is clearly depicted in the histogram of the target variable (Figure 1). Similar skewness was also evident in the distributions of many other features (Figure 2). Descriptive statistics for `shares` highlight this challenge: a mean of approximately 3349, a median of 1400, a 75th percentile of 2700, and a maximum exceeding $663,000$, indicating a wide range and the presence of numerous outliers.

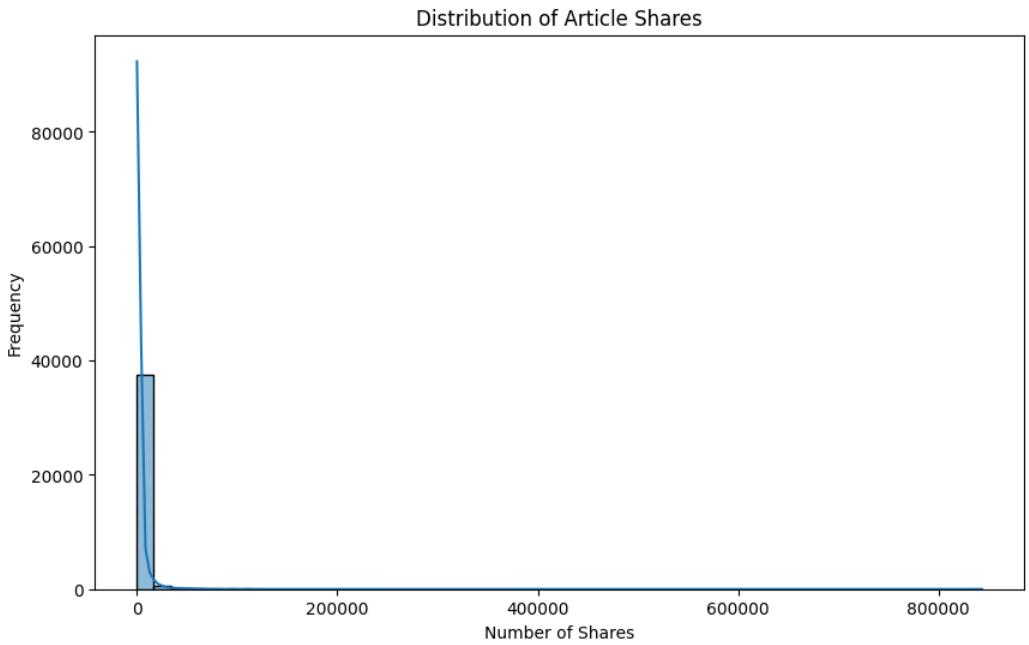

Figure 1: Distribution of Shares

Temporal analysis showed distinct patterns in publication frequency and popularity. Article publication is highest on weekdays, particularly Wednesday and Tuesday, with significantly fewer articles published on weekends (Figure 4). Correspondingly, the total number of shares broadly align with this, with weekdays accumulating more shares than individual weekend days (Figure 5).

An examination of content categories revealed that the 'World', 'Tech', 'Entertainment', and 'Business' channels published the most articles (Figure 6). However, in terms of total shares, the 'Tech' and 'Entertainment' lead in total shares, suggesting higher average popularity in these categories compared to others with similar or higher publication rates (Figure 7). LDA topic analysis also indicated that the dominant topic is associated with varying average share counts (Figure 8).

A correlation heatmap (Figure 9) provided an overview of feature relationships. While some features were correlated with each other, linear correlations between most individual features and the target variable (`shares`) were generally low. Keyword statistics (e.g., `kw_max_avg`, `kw_avg_avg`) and self-reference shares (`self_reference_avg_sharess`) showed some of the relatively stronger positive linear relationships, though these were still moderate. Scatter plots of textual features and topic purity against shares further illustrated the complex, often non-linear, relationship with popularity, particularly for high-share articles (Figure 10, Figure 11).

Overall, the EDA revealed the inherent challenges of predicting the exact number of shares, primarily due to the target variable's severe skewness and the complex, often non-linear, relationships between features and popularity.

2.5 Methodology

The project followed a structured methodology for training and evaluating regression models to predict online news popularity.

2.5.1 Data Preprocessing and Feature Engineering

The initial data loading and inspection involved handling any duplicate entries and ensuring correct data types. The non-predictive `url` and `timedelta` features were removed. Addressing the observed data skewness in many features and the target variable was a critical step. To explore different strategies

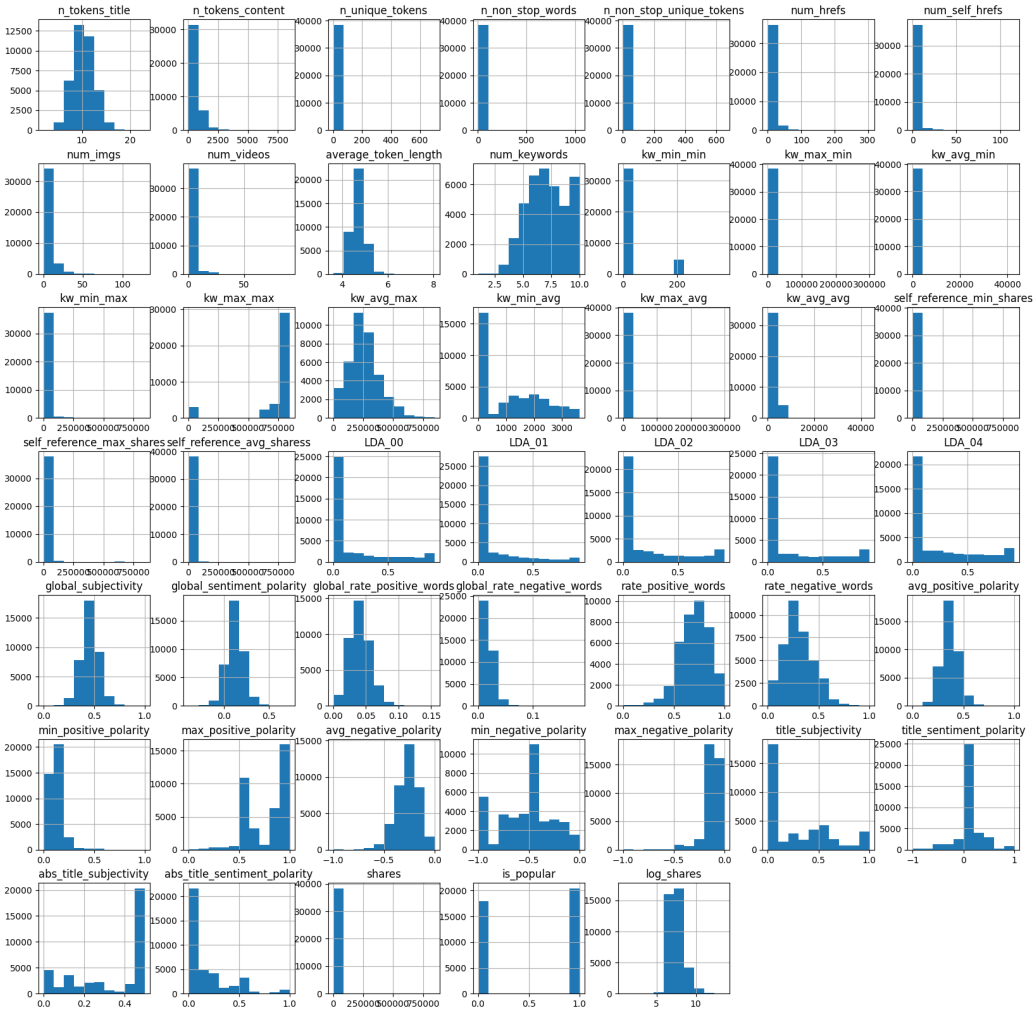

Figure 2: Distribution of Various Features

for handling skewness and compare their impact on model performance, both Log transformation (specifically $\log(x+1)$ for positive data) and Yeo-Johnson transformation were applied to the relevant skewed features and the target variable. The Yeo-Johnson transformation was particularly useful as it can normalize data regardless of positive or negative values. To handle the influence of outliers, which were prevalent, **Robust Scaling** was applied to the features. Robust Scaling, based on quartiles, is less sensitive to extreme values than standard scaling methods.

Principal Component Analysis (PCA) was also applied to the features. PCA is a dimensionality reduction technique used to transform the features into a smaller set of uncorrelated components while retaining most of the variance in the data. This step was likely performed to potentially reduce the dimensionality of the feature space and address multicollinearity observed in the correlation heatmap, preparing the data for the regression models.

Furthermore, several new features were engineered to potentially provide more informative inputs to the models by combining existing sentiment and polarity-related features:

- `sentiment_balance` = `global_rate_positive_words` - `global_rate_negative_words` (Net sentiment).
- `sentiment_intensity` = `global_rate_positive_words` + `global_rate_negative_words` (Overall emotional language prominence).
- `pos_polarity_strength` = (`max_positive_polarity` - `min_positive_polarity`) * `avg_positive_polarity` (Range and intensity of positive sentiment polarity).

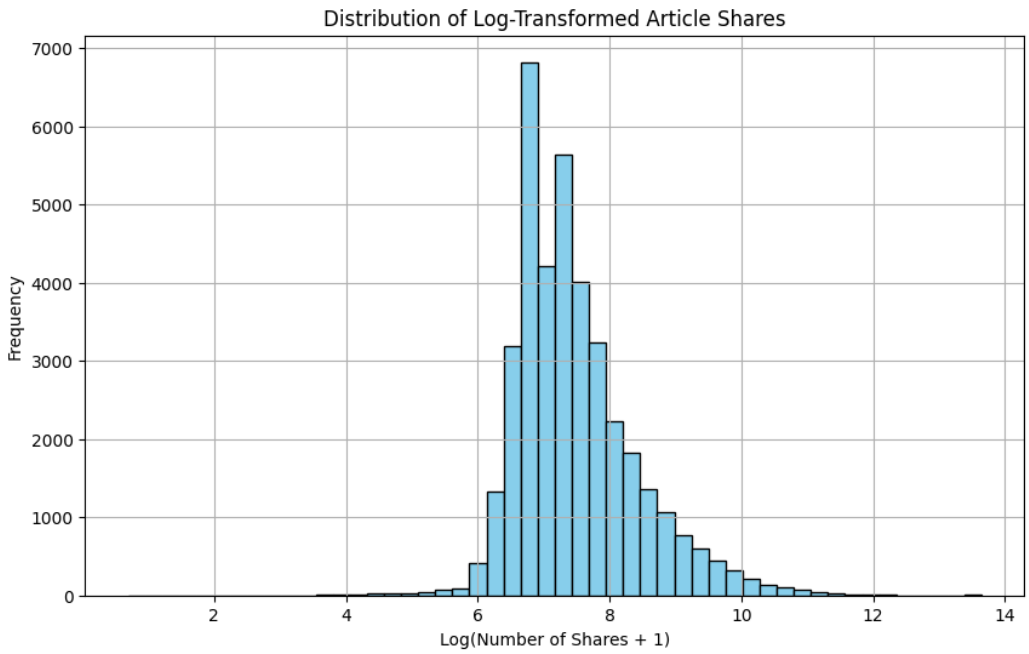

Figure 3: Distribution of Shares After Log Transformation

- `neg_polarity_strength` = (`max_negative_polarity` - `min_negative_polarity`) * `avg_negative_polarity` (Range and intensity of negative sentiment polarity).
- `title_impact` = `abs_title_sentiment_polarity` * `title_subjectivity` (Strength of the title's emotional tone and opinionatedness).
- `title_bias` = `title_sentiment_polarity` / (`abs_title_sentiment_polarity` + 10^{-6}) (with a small constant to prevent division by zero).

These engineered features were added to the dataset prior to modeling.

2.5.2 Data Splitting

The prepared dataset, consisting of features and the continuous target variable (`shares`), was split into three sets: training, validation, and test, to ensure robust model evaluation. This resulted in a final distribution of approximately 60% for training, 20% for validation, and 20% for testing. The training set was used for model training, the validation set for hyperparameter tuning and model selection, and the test set for final evaluation. The split was performed with `random_state=42` for reproducibility.

2.5.3 Model Selection and Training

For this project, seven different regression algorithms were implemented and compared to evaluate their performance in predicting online news shares: Linear Regression, Bayesian Ridge, K-Nearest Neighbors (KNN), Support Vector Regressor (SVM), Decision Tree Regressor, Random Forest Regressor, and Neural Network. These models were trained on the preprocessed training data. Hyperparameter tuning was performed for some models, such as SVR, using technique GridSearchCV with cross-validation on the training set and evaluation on the validation set (example GridSearchCV output from Figure 14 shows tuning for SVR).

2.5.4 Model Evaluation

The performance of each trained model was evaluated using four standard regression metrics:

- Mean Squared Error (MSE)

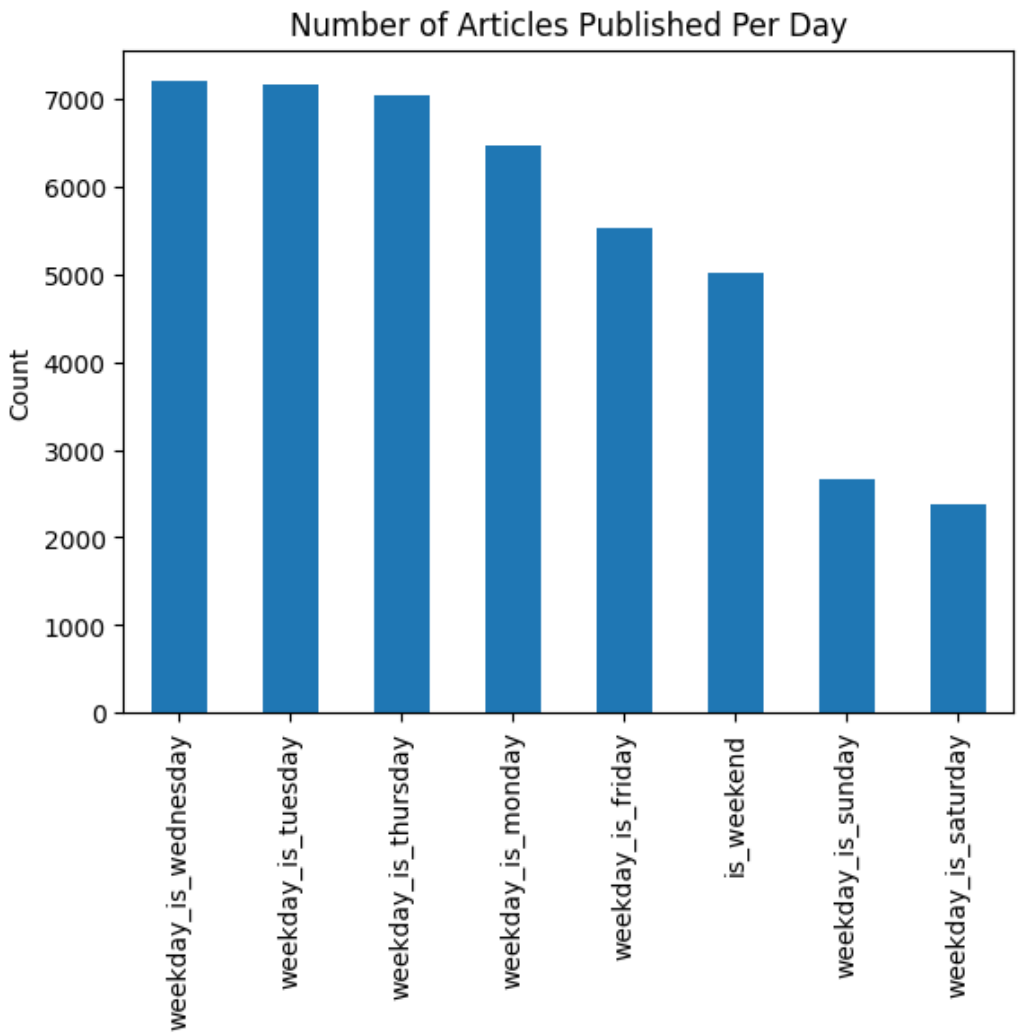

Figure 4: Number of Articles Published Per Day

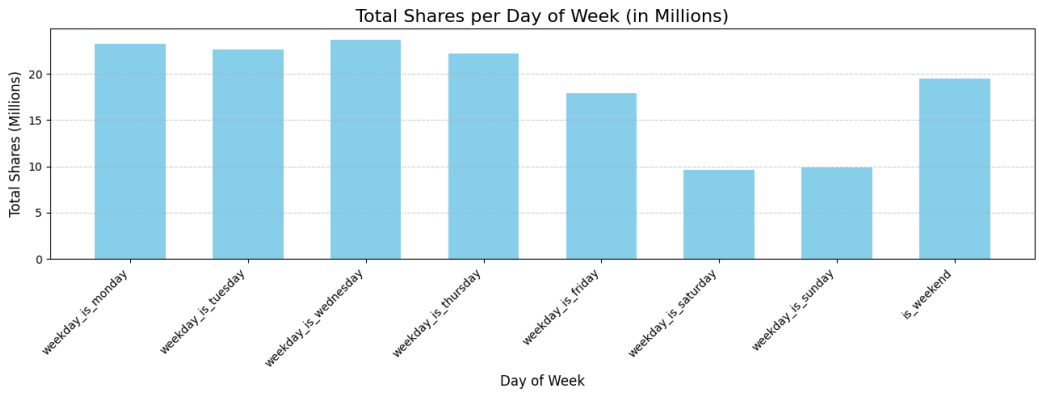

Figure 5: Total Shares Per Day of Week

- Root Mean Squared Error (RMSE)
- Mean Absolute Error (MAE)
- R-squared (R^2)

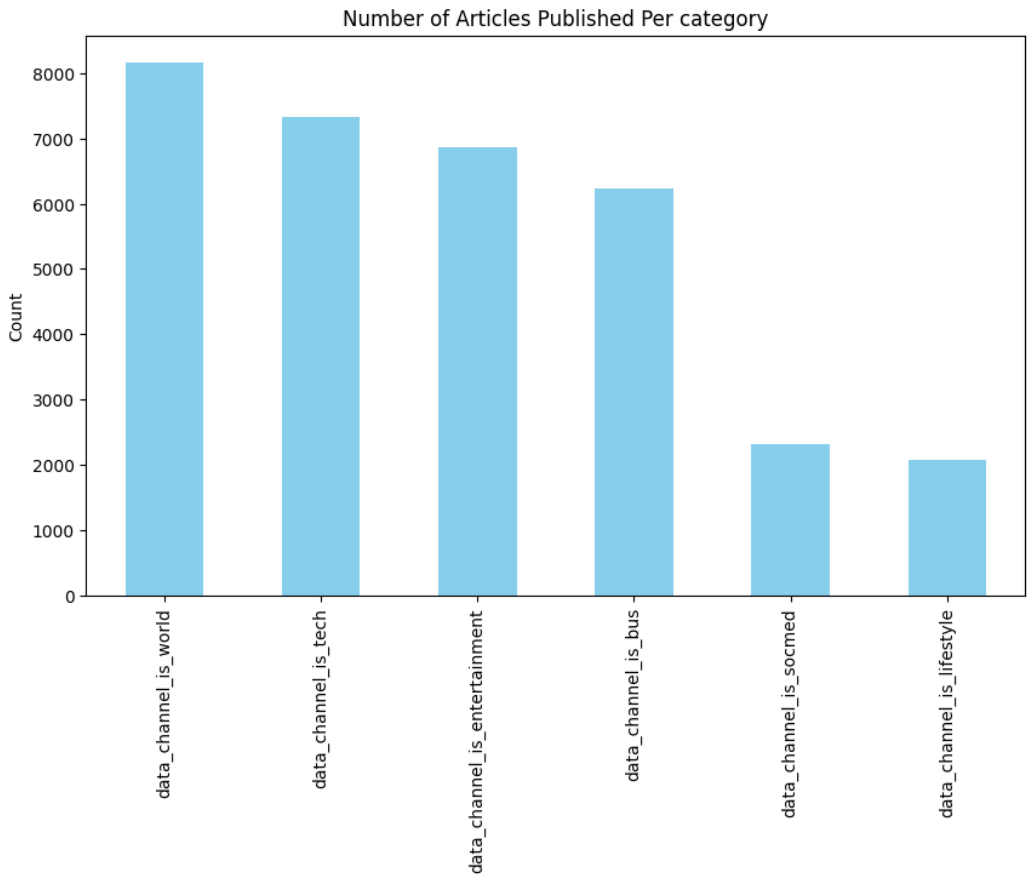

Figure 6: Number of Articles Published Per Category

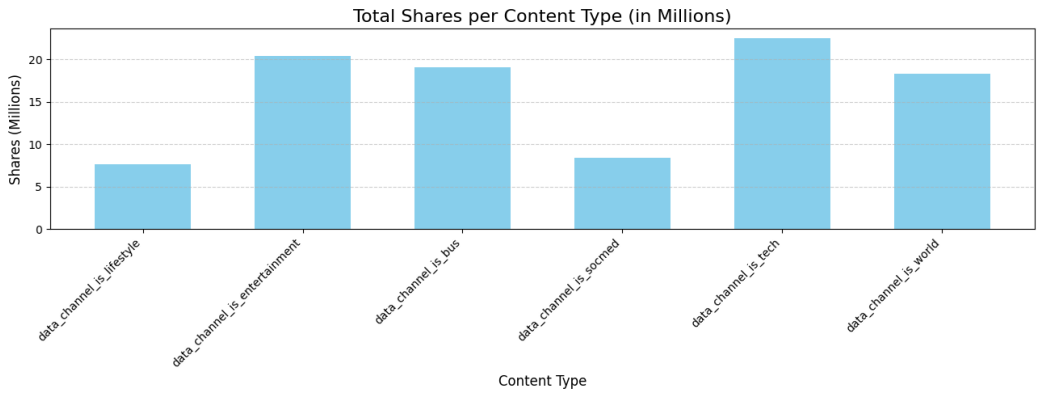

Figure 7: Total Shares Per Content Type

Models were evaluated on both the validation and test sets, with the test set evaluation providing the final assessment of the model's generalization ability.

2.6 Results

The regression models were evaluated on the test set using MAE, MSE, RMSE, and R^2 metrics. To understand the impact of different skewness handling approaches, performance was assessed for models trained using data prepared with Log transformation versus Yeo-Johnson transformation, and evaluated

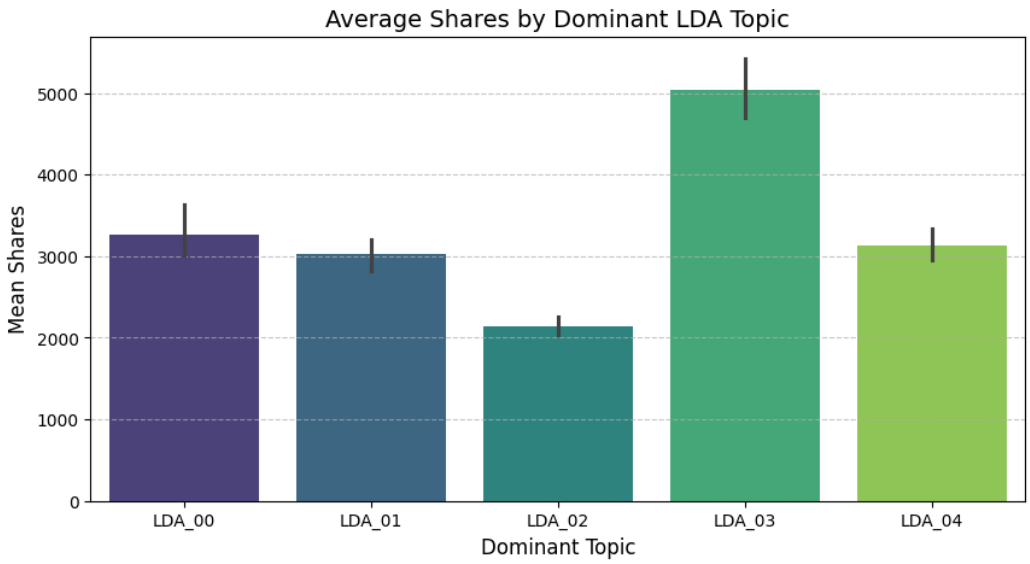

Figure 8: Average Shares by Dominant LDA Topic

on both the transformed and original target scales.

2.6.1 Performance on Transformed Scales

When evaluating the models using MSE and R^2 directly on the log-transformed shares target (using the results from Figure 12), the error magnitudes were significantly smaller compared to the original scale. For example, MSE values ranged from approximately 0.74 to 1.48, and some models achieved positive R^2 values, albeit low ones (e.g., Random Forest at 0.14, Bayesian Ridge and Linear Regression at 0.13, SVM at 0.11). This suggests that the models were able to capture some variance in the transformed target space, which has a more normalized distribution.

2.6.2 Performance on Original Scale (after Transformation)

Evaluating the models on the original target scale after training with Yeo-Johnson transformed data provides a more direct measure of performance in predicting the actual number of shares. Table 3 presents these results.

Table 3: Regression Model Performance on the Test Set (Original Shares Scale) after Yeo-Johnson Transformation and Robust Scaling

Model	Test MAE	Test MSE	Test RMSE	Test R^2
SVM	2321.16	1.274784×10^8	11290.63	-0.00751
Random Forest	2341.60	1.274289×10^8	11288.44	-0.00712
Bayesian Ridge	2358.19	1.280922×10^8	11317.78	-0.01236
Linear Regression	2359.34	1.281673×10^8	11321.09	-0.01296
KNN	2396.89	1.655150×10^8	12865.26	-0.00777
Neural Network	2421.32	1.278625×10^8	11307.63	-0.01055
Decision Tree	4051.12	3.375524×10^8	18372.60	-1.66781

As observed from the results in Table 3 and visually represented in the bar charts (Figure 13), the **Decision Tree Regressor performed significantly worse** than all other models across all metrics

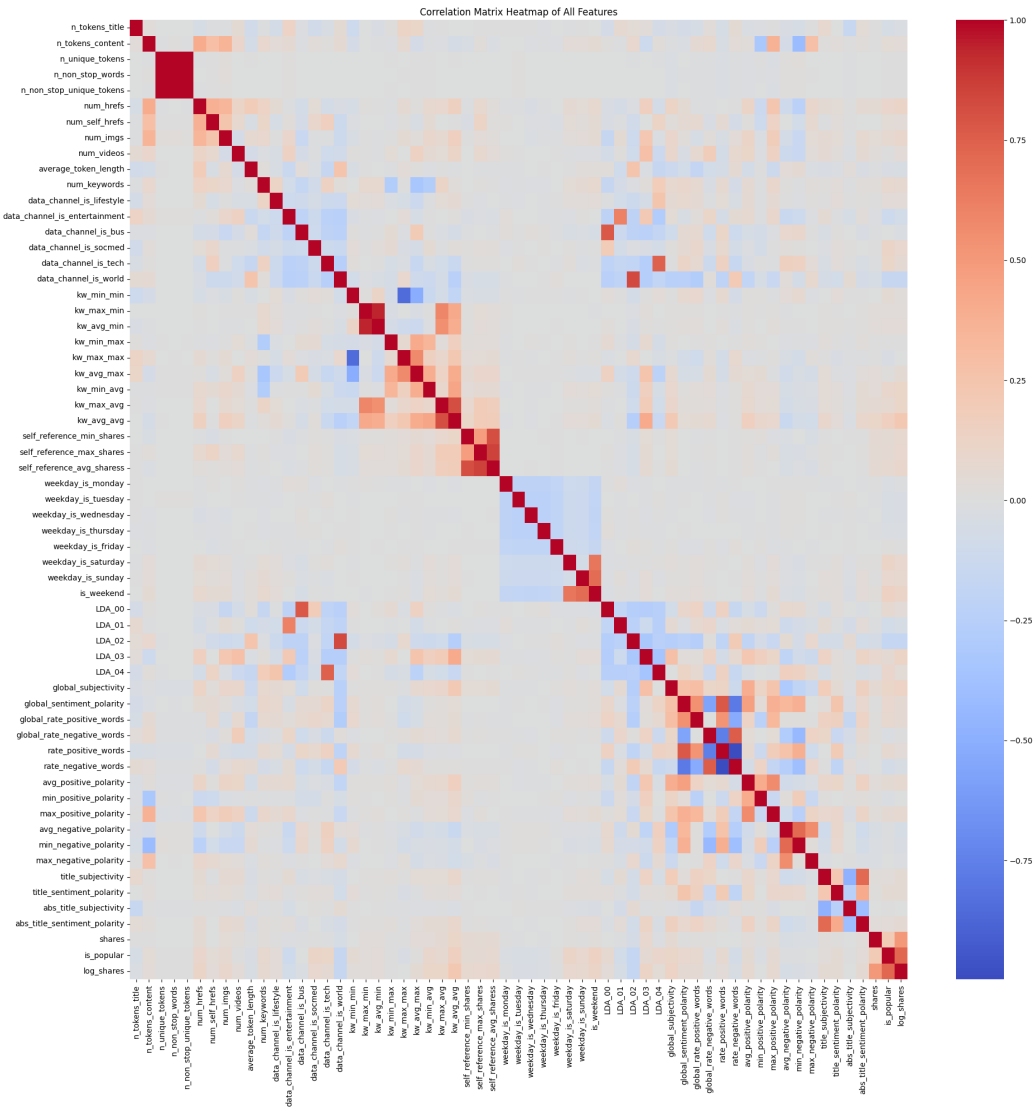

Figure 9: Correlation Matrix Heatmap of All Features

(MAE, MSE, RMSE, R^2). Its errors were roughly twice as high as the next worst models, and its R^2 value was highly negative.

Among the remaining six models, performance was relatively similar, although some minor distinctions can be made:

- **SVM** achieved the lowest Mean Absolute Error (MAE) at 2321.16, suggesting it had the smallest average prediction error magnitude.
- **Random Forest** and **SVM** had the lowest Mean Squared Error (MSE) and Root Mean Squared Error (RMSE), which are more sensitive to larger errors. Random Forest had a slightly lower MSE (1.274289×10^8) and RMSE (11288.44) than SVM (MSE 1.274784×10^8, RMSE 11290.63).
- **Neural Network** and **KNN** generally showed slightly higher MAE compared to SVM, Random Forest, Bayesian Ridge, and Linear Regression within this group. KNN also had noticeably higher MSE and RMSE than the other five non-Decision Tree models.
- All six models (excluding Decision Tree) exhibited very low or negative R^2 values, ranging from approximately -0.013 to -0.007.

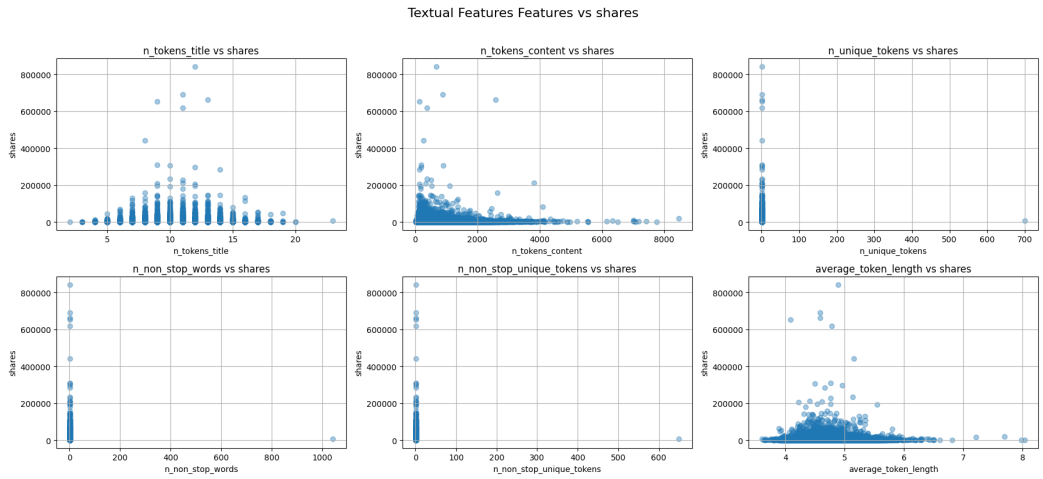

Figure 10: Textual Features vs Shares

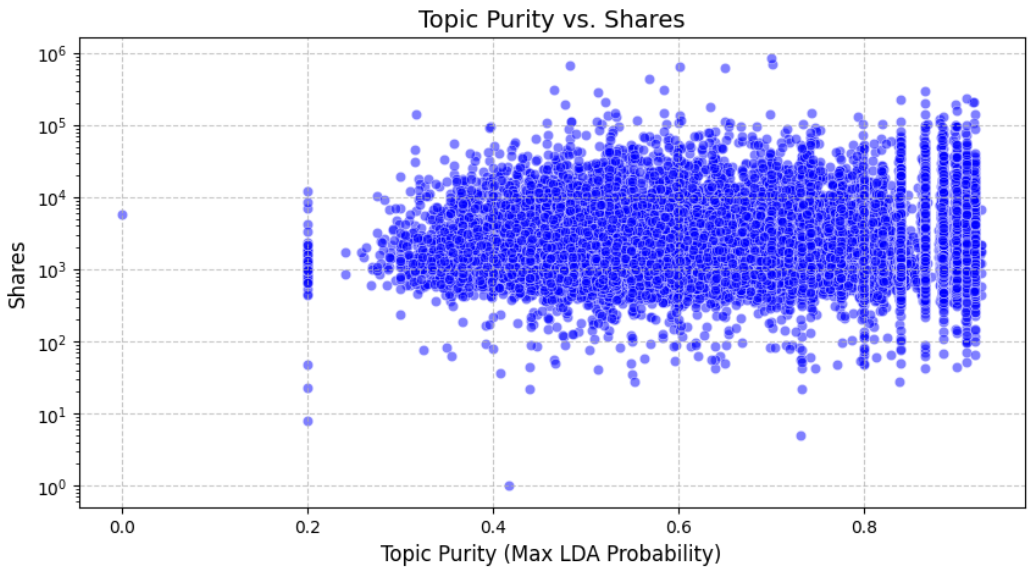

Figure 11: Topic Purity vs Shares

```
Decision Tree: MSE = 1.48, R2 = -0.72
Random Forest: MSE = 0.74, R2 = 0.14
KNN: MSE = 0.87, R2 = -0.02
Bayesian Ridge: MSE = 0.75, R2 = 0.13
SVM: MSE = 0.76, R2 = 0.11
Neural Network: MSE = 0.86, R2 = -0.01
Linear Regression: MSE = 0.75, R2 = 0.13
```

Figure 12: Model Performance Metrics on Log-Transformed Scale

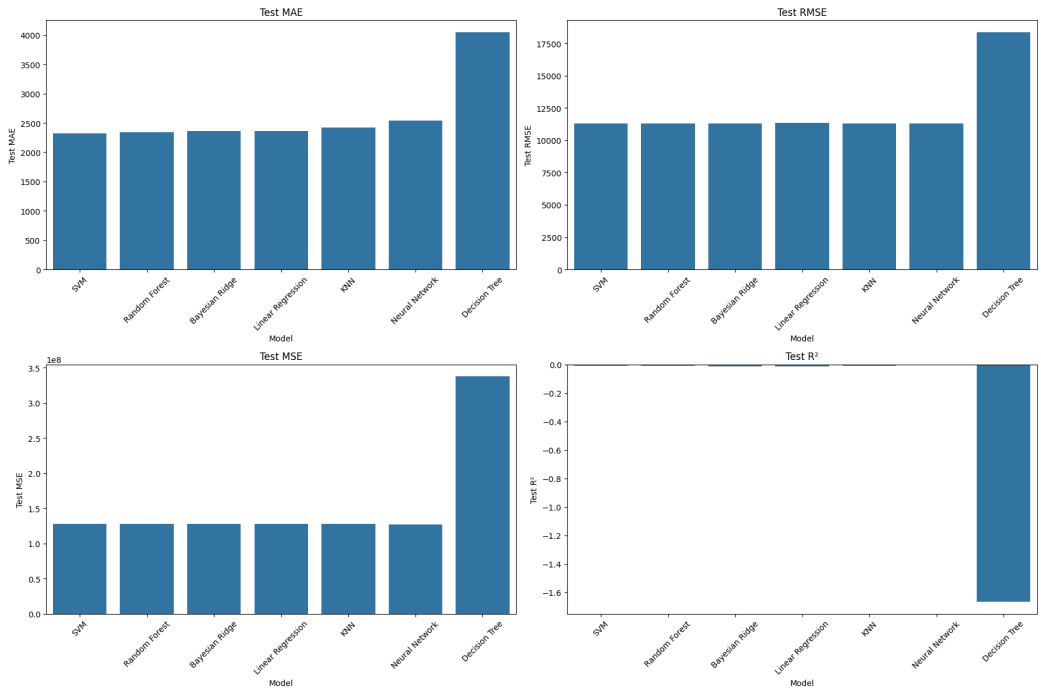

Figure 13: Test Set Performance Metrics (Original Scale)

2.6.3 Hyperparameter Tuning for SVR

As part of the model development, hyperparameter tuning was performed for the Support Vector Regressor using GridSearchCV with cross-validation. Tuning was explored using data prepared with both Log and Yeo-Johnson transformations to identify optimal parameters under different data distributions.

When trained and evaluated on data processed with **Yeo-Johnson transformation**, GridSearchCV identified the best SVR parameters as {`'C'`: 0.1, `'epsilon'`: 0.2, `'kernel'`: `'rbf'`}. The performance metrics for this tuned model on the validation and test sets (e.g., Test MAE $\approx$ 2317.4, Test RMSE $\approx$ 11324.4) align closely with the SVR results presented in Table 3, confirming its performance when trained with Yeo-Johnson transformed data and evaluated on the original scale.

Tuning was **also performed using data prepared with Log transformation** (Figure 14). For example, tuning for an SVR with a 'linear' kernel resulted in a Validation MAE of approximately 2317.5 (when evaluated on the original scale).

```
Fitting 5 folds for each of 18 candidates, totalling 90 fits
🔧 Best SVR Parameters: {'C': 10, 'epsilon': 0.1, 'kernel': 'linear'}
✅ Best CV MAE: 2322.0502
Validation MAE: 2317.4743
```

Figure 14: Example GridSearchCV Output for SVR Tuning (Log Transformation)

These tuning efforts demonstrate that while hyperparameter optimization was conducted for SVR under different transformation strategies, the resulting models consistently showed similar performance levels when evaluated on the original shares scale. This further reinforces the observation that accurately predicting the precise, highly variable share count remains a significant challenge regardless of the specific transformation or SVR parameters used.

Despite the minor differences among the top six models (SVM and Random Forest showing a slight edge in minimizing error metrics), their overall performance was poor. The consistently low and negative R^2 values across virtually all models (including those with the "best" MAE/RMSE) indicate

that none of the implemented algorithms were able to explain a significant portion of the variance in the actual number of shares.

Comparing the error metrics (MAE, RMSE) on the original scale to the descriptive statistics of the target variable (mean ≈ 3349, median ≈ 1400, 75th percentile ≈ 2700), an MAE of around $2300 - 2500$ shares is substantial. It suggests that, on average, the models' predictions were off by an amount comparable to or exceeding the typical share counts for most articles. The large RMSE values further emphasize that larger errors were frequent, highlighting the difficulty in predicting high share counts.

The scatter plot of Actual vs. Predicted shares for the SVM model (Figure 15) visually confirms the models' limitations on the original scale. It shows that even the model with the lowest MAE primarily predicts within a relatively narrow range of low share counts, failing to accurately predict the articles with high or very high numbers of shares. This inability to capture and predict the viral content, which contributes significantly to the overall variance and the long tail of the distribution, is the main reason for the high error metrics and the low/negative R^2 values when evaluating on the original scale.

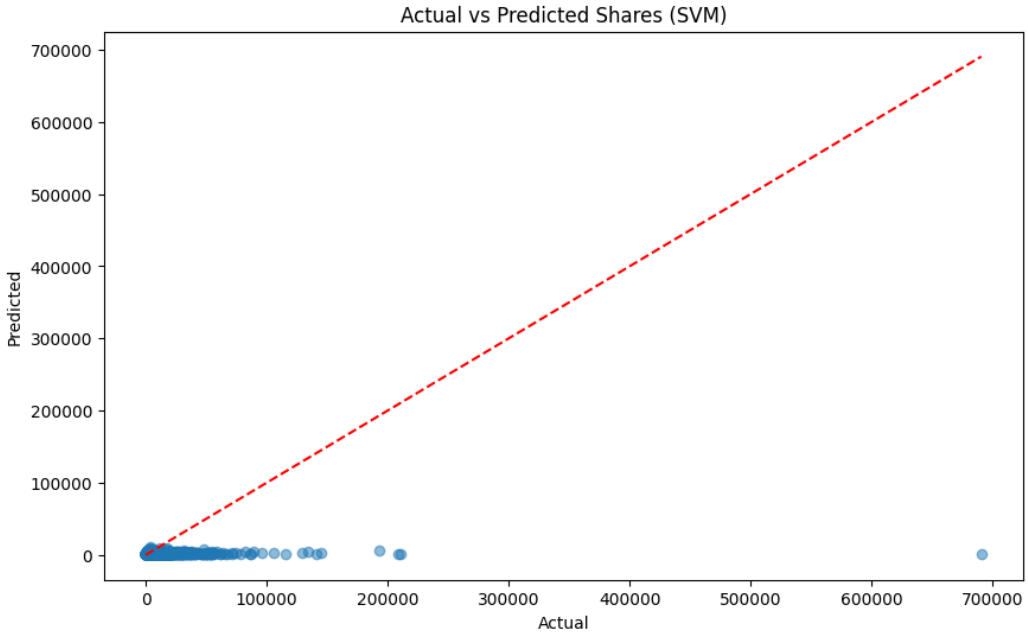

Figure 15: Actual vs. Predicted Shares for SVM (Original Scale)

2.7 Conclusion

In conclusion, while different data transformation techniques (Log and Yeo-Johnson) were explored to handle skewness and showed some ability to model the transformed target, the evaluation on the original shares scale revealed significant challenges. The models struggled to effectively predict the precise number of shares, particularly for high-popularity articles. While SVM and Random Forest showed marginally better performance in minimizing error metrics like MAE and RMSE among the evaluated models (excluding the poor-performing Decision Tree), the consistently high error magnitudes relative to the typical share counts and the very low/negative R^2 values indicate that, based on the evaluated models and methodology, the data is not well-suited for accurately predicting the precise popularity score as a regression problem using these standard techniques. Given the significant difficulty in predicting the exact share count, this dataset might be more amenable to a classification task, such as predicting whether an article will be popular (e.g., above median shares) or not popular.

References

[1] g-shreekant. (n.d.). Heart-Disease-Prediction. *GitHub*. Available at: https://github.com/kb22/Heart-Disease-Prediction/blob/master/Heart%20Disease%20Prediction.ipynb

[2] (Author not specified). (Date not specified). Heart Disease Prediction. *Towards Data Science*. Available at: https://towardsdatascience.com/heart-disease-prediction-73468d630cfc/

[3] Centers for Disease Control and Prevention. (n.d.). About Heart Disease. *CDC*. Available at: https://www.cdc.gov/heart-disease/about/index.html

[4] UCI Machine Learning Repository. (n.d.). Heart Disease Dataset. Available at: https://archive.ics.uci.edu/dataset/45/heart+disease

[5] Szymanowski, J. (2023). Predicting the Popularity of an Online News Article. *Medium*. Available at: https://joshua-szymanowski.medium.com/predicting-the-popularity-of-an-online-news-article-5b07591146bf

[6] Balaji, S. (n.d.). ML Capstone Project. *GitHub Pages*. Available at: https://swebalaji.github.io/ML_Capstone_Project/

[7] Rangarajan, S. (n.d.). Online-News-Popularity-Regression. *GitHub*. Available at: https://github.com/Sushama-Rangarajan/Online-News-Popularity-Regression?tab=readme-ov-file

Who did what in the project?

- UCI Heart Disease classification project done by Khaldi Aya and Benbouabdellah Nadia.
- Online News popularity done by Abdeli Zohra and Hadj Messaouad Maria.